

Chapter 4 (Classification) — Solutions

An Introduction to Statistical Learning

Problem 1 — Equivalence of (4.2) and (4.3)

The logistic-function form (4.2) is

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}},$$

and the logit (odds) form (4.3) is

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0 + \beta_1 X}.$$

Proof. Start from (4.2) and compute $1 - p(X)$:

$$1 - p(X) = 1 - \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} = \frac{(1 + e^{\beta_0 + \beta_1 X}) - e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} = \frac{1}{1 + e^{\beta_0 + \beta_1 X}}.$$

Now form the odds by dividing:

$$\frac{p(X)}{1 - p(X)} = \frac{\frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}}{\frac{1}{1 + e^{\beta_0 + \beta_1 X}}} = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} \cdot (1 + e^{\beta_0 + \beta_1 X}) = e^{\beta_0 + \beta_1 X}.$$

The common factor $1 + e^{\beta_0 + \beta_1 X}$ cancels, leaving exactly (4.3). Hence the two representations are equivalent. ■

Taking logs of both sides also gives the familiar *log-odds* (logit) form, $\log(p(X)/(1 - p(X))) = \beta_0 + \beta_1 X$, which is linear in X .

Problem 2 — Logistic regression for grades

Fitted model: $\hat{\beta}_0 = -6$, $\hat{\beta}_1 = 0.05$ (hours studied X_1), $\hat{\beta}_2 = 1$ (undergrad GPA X_2), so

$$\hat{p}(X) = \frac{e^{-6 + 0.05X_1 + X_2}}{1 + e^{-6 + 0.05X_1 + X_2}}.$$

(a) For a student with $X_1 = 40$ hours and $X_2 = 3.5$:

$$\text{logit} = -6 + 0.05(40) + 1(3.5) = -6 + 2 + 3.5 = -0.5.$$

$$\hat{p} = \frac{e^{-0.5}}{1 + e^{-0.5}} = \frac{0.6065}{1.6065} \approx \boxed{0.378}.$$

So the estimated probability of getting an A is about **37.8%**.

(b) A 50% chance means $\hat{p} = 0.5$, i.e. the log-odds equal 0:

$$-6 + 0.05X_1 + 3.5 = 0 \implies 0.05X_1 = 2.5 \implies X_1 = \boxed{50}.$$

With a GPA of 3.5, the student would need to study **50 hours** for a 50% chance of an A.

Problem 3 — Detailed forms of a_k , b_{kj} , b_{kjl} in (4.33)

Under QDA, $X | Y = k \sim \mathcal{N}(\mu_k, \Sigma_k)$, with density

$$f_k(x) = (2\pi)^{-p/2} |\Sigma_k|^{-1/2} \exp\left(-\frac{1}{2}(x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k)\right).$$

By Bayes' theorem $\Pr(Y = k | X = x) \propto \pi_k f_k(x)$. Comparing class k to the baseline class K via the log-ratio:

$$\log \frac{\Pr(Y = k | x)}{\Pr(Y = K | x)} = \log \frac{\pi_k}{\pi_K} + \log f_k(x) - \log f_K(x).$$

Using $\log f_k(x) = -\frac{p}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k)$, the $-\frac{p}{2} \log(2\pi)$ terms cancel. Expanding each quadratic form,

$$(x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k) = x^\top \Sigma_k^{-1} x - 2\mu_k^\top \Sigma_k^{-1} x + \mu_k^\top \Sigma_k^{-1} \mu_k,$$

and collecting the constant, linear, and quadratic terms gives the form of (4.33):

$$\log \frac{\Pr(Y = k | x)}{\Pr(Y = K | x)} = a_k + \sum_{j=1}^p b_{kj} x_j + \sum_{j=1}^p \sum_{l=1}^p b_{kjl} x_j x_l,$$

with

$$a_k = \log \frac{\pi_k}{\pi_K} - \frac{1}{2} \log \frac{|\Sigma_k|}{|\Sigma_K|} - \frac{1}{2} \mu_k^\top \Sigma_k^{-1} \mu_k + \frac{1}{2} \mu_K^\top \Sigma_K^{-1} \mu_K$$

$$b_{kj} = (\Sigma_k^{-1} \mu_k - \Sigma_K^{-1} \mu_K)_j \quad b_{kjl} = -\frac{1}{2} [(\Sigma_k^{-1})_{jl} - (\Sigma_K^{-1})_{jl}]$$

where $(\cdot)_j$ denotes the j -th entry of a vector and $(\cdot)_{jl}$ the (j, l) entry of a matrix.

Remark. The quadratic coefficients b_{kjl} are nonzero *because* Σ_k *differs across classes* — this is what makes the QDA decision boundary quadratic. If all classes share a common covariance ($\Sigma_k = \Sigma$ for every k), then $\Sigma_k^{-1} - \Sigma_K^{-1} = 0$, so every $b_{kjl} = 0$ and the expression collapses to the *linear* LDA discriminant.

Problem 4 — Predicting gas mileage on the Auto data

(a) Create the binary response mpg01.

```
library(ISLR2); library(MASS); library(e1071); library(class)
attach(Auto)
mpg01 <- ifelse(mpg > median(mpg), 1, 0)
Auto <- data.frame(Auto, mpg01)
```

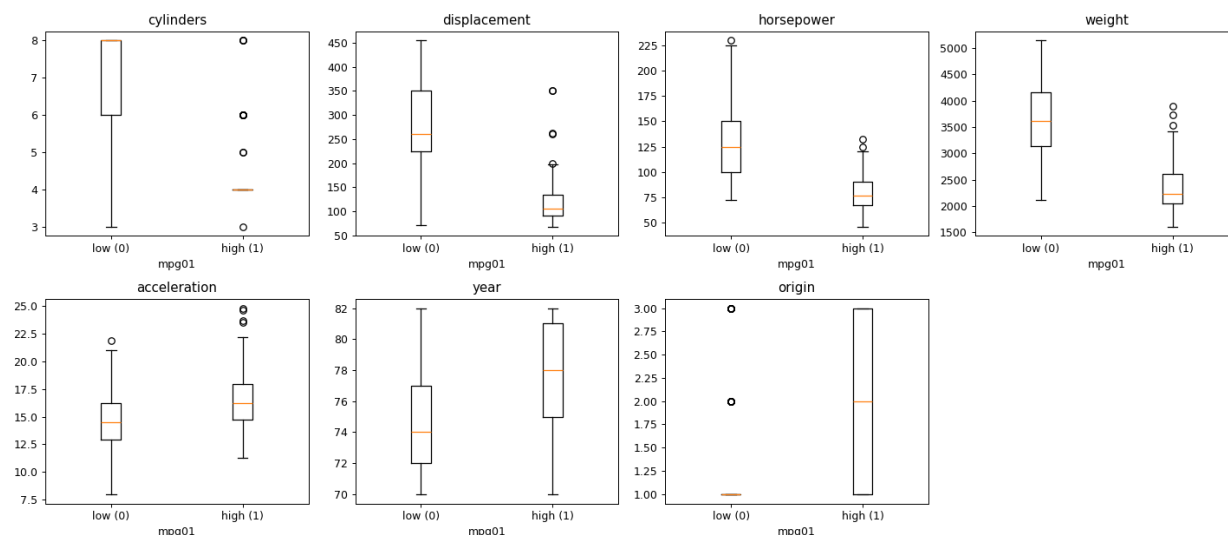
The median mpg is 22.75, giving a perfectly balanced response (196 in each class).

(b) Explore associations.

```
cor(subset(Auto, select = -name)) # correlations with mpg01
par(mfrow = c(2,3))
for (v in c("cylinders", "displacement", "horsepower",
            "weight", "acceleration", "year"))
  boxplot(Auto[[v]] ~ mpg01, main = v, xlab = "mpg01")
```

Correlations of each feature with mpg01:

	cylinders	weight	displacement	horsepower	origin	year	acceleration
corr with mpg01	−0.759	−0.758	−0.753	−0.667	0.514	0.430	0.347



Findings. cylinders, weight, displacement, and horsepower are all strongly *negatively* associated with mpg01: high-mileage cars tend to have fewer cylinders and lower engine displacement, horsepower, and weight, and their boxplots show clear separation between the two classes with little overlap. acceleration, year, and origin are more weakly associated and overlap substantially. We therefore use the four strong predictors — **cylinders, displacement, horsepower, weight** — in the models below.

(c) **Train/test split** (70%/30%, stratified, seed 1).

```
set.seed(1)
train <- sample(nrow(Auto), 0.7 * nrow(Auto))
Auto.train <- Auto[train, ]; Auto.test <- Auto[-train, ]
mpg01.test <- Auto.test$mpg01
f <- mpg01 ~ cylinders + displacement + horsepower + weight
```

This yields 274 training and 118 test observations.

(d) **LDA.**

```
lda.fit <- lda(f, data = Auto.train)
lda.pred <- predict(lda.fit, Auto.test)$class
mean(lda.pred != mpg01.test) # test error
```

Test error $\approx 10.2\%$.

(e) **QDA.**

```
qda.fit <- qda(f, data = Auto.train)
qda.pred <- predict(qda.fit, Auto.test)$class
mean(qda.pred != mpg01.test)
```

Test error $\approx 11.0\%$.

(f) Logistic regression.

```
glm.fit <- glm(f, data = Auto.train, family = binomial)
glm.prob <- predict(glm.fit, Auto.test, type = "response")
glm.pred <- ifelse(glm.prob > 0.5, 1, 0)
mean(glm.pred != mpg01.test)
```

Test error $\approx 11.0\%$.

(g) Naive Bayes.

```
nb.fit <- naiveBayes(f, data = Auto.train)
nb.pred <- predict(nb.fit, Auto.test)
mean(nb.pred != mpg01.test)
```

Test error $\approx 11.0\%$.

(h) KNN (predictors standardized; several values of K).

```
vars <- c("cylinders", "displacement", "horsepower", "weight")
train.X <- scale(Auto.train[, vars])
test.X <- scale(Auto.test[, vars],
               center = attr(train.X, "scaled:center"),
               scale = attr(train.X, "scaled:scale"))
train.Y <- Auto.train$mpg01
for (k in c(1, 3, 5, 10, 15, 20, 50, 100)) {
  set.seed(1)
  pred <- knn(train.X, test.X, train.Y, k = k)
  cat("K =", k, " test error =", mean(pred != mpg01.test), "\n")
}
```

K	1	3	5	10	15	20	50	100
test error	9.3%	11.0%	9.3%	11.0%	11.0%	11.0%	11.0%	11.0%

The best KNN performance ($\approx 9.3\%$) occurs at small K ($K = 1$ or 5); performance is stable and slightly worse for larger K .

Summary

Method	Test error
LDA	10.2%
QDA	11.0%
Logistic regression	11.0%
Naive Bayes	11.0%
KNN (best, $K = 1$ or 5)	9.3%

All five methods perform similarly, with test errors clustered around 9–11%; KNN at small K and LDA are marginally best. The strong, roughly linear separation provided by cylinders, displacement, horsepower, and weight explains why even simple linear methods do well.

Note. The reported test errors come from a 70/30 stratified split (`set.seed(1)`). Exact figures depend on the particular split and the random-number generator, so your numbers may differ by a point or two, but the ranking and overall $\sim 10\%$ error level are robust.